04. 네트워킹 시스템 분석

개요

Reth의 네트워킹 시스템은 Ethereum의 devp2p 프로토콜을 구현하여 P2P 네트워크 통신을 담당합니다. 이 시스템은 피어 발견(Peer Discovery), 연결 관리, 메시지 전송/수신을 통해 분산 네트워크에서 노드 간 통신을 가능하게 합니다.

핵심 구조체 및 트레이트

NetworkHandle

네트워크와 상호작용하기 위한 메인 인터페이스입니다.

```
#[derive(Clone, Debug)]
pub struct NetworkHandle<N: NetworkPrimitives = EthNetworkPrimitives> {
inner: Arc<NetworkInner<N>>,
}

impl<N: NetworkPrimitives> NetworkHandle<N> {
/// 피어에게 요청을 전송
pub fn send_request(&self, peer_id: PeerId, request: PeerRequest<N>) {
self.send_message(NetworkHandleMessage::EthRequest { peer_id, request })
}
```

```
/// 트랜잭션 해시를 피어에게 전송
pub fn send_transactions_hashes(&self, peer_id: PeerId, msg: NewPooledTransact
self.send_message(NetworkHandleMessage::SendPooledTransactionHashes { peer_id, m
}

/// 전체 트랜잭션을 피어에게 전송
pub fn send_transactions(&self, peer_id: PeerId, msg: Vec<Arc<N::BroadcastedTr
self.send_message(NetworkHandleMessage::SendTransaction {
peer_id,
msg: SharedTransactions(msg),
})
}
}
```

NetworkManager

전체 네트워크 상태를 관리하는 핵심 컴포넌트입니다.

```
#[derive(Debug)]
pub struct NetworkManager<N: NetworkPrimitives = EthNetworkPrimitives> {
/// 실제 네트워크 부분을 관리하는 타입 (연결 포함)
swarm: Swarm<N>,
/// 공유 가능한 네트워크 핸들
```

```
handle: NetworkHandle<N>,

/// 명령 채널의 수신부

from_handle_rx: UnboundedReceiverStream<NetworkHandleMessage<N>>,

/// eth 프로토콜에 따른 블록 가져오기 처리

block_import: Box<dyn BlockImport<N::NewBlockPayload>>,

/// 고수준 네트워크 이벤트 전송자

event_sender: EventSender<NetworkEvent<PeerRequest<N>>>,

/// 활성 피어 세션 수 추적

num_active_peers: Arc<AtomicUsize>,

}
```

피어 발견 프로토콜

Discovery v4 (Discv4)

Kademlia 기반의 DHT를 사용하는 피어 발견 프로토콜입니다.

```
#[derive(Debug, Clone)]

pub  struct Discv4 {

/// UDP 소켓의 주소

local_addr: SocketAddr,

/// 서비스에 명령을 보내는 채널

to_service: mpsc::UnboundedSender<Discv4Command>,

/// 로컬 노드 레코드 추적
```

```
node_record: Arc<Mutex<NodeRecord>>,
}

impl Discv4 {
/// 자신을 찾는 FindNode 재귀 조회 시작
pub  async  fn  lookup_self(&self) -> Result<Vec<NodeRecord>, Discv4Error> {
self.lookup_node(None).await
}

/// 주어진 노드 ID를 조회
pub  async  fn  lookup(&self, node_id: PeerId) -> Result<Vec<NodeRecord>, Discv4
self.lookup_node(Some(node_id)).await
}

/// 피어를 테이블에서 제거
pub  fn  remove_peer(&self, node_id: PeerId) {
let  cmd = Discv4Command::Remove(node_id);
self.send_to_service(cmd);
}

/// 노드를 테이블에 추가
pub  fn  add_node(&self, node_record: NodeRecord) {
let  cmd = Discv4Command::Add(node_record);
```

```
self.send_to_service(cmd);
}
}
```

Discovery v5 (Discv5)

더 향상된 피어 발견 프로토콜로, ENR(Ethereum Node Record)을 지원합니다.

```
#[derive(Clone)]
pub struct Discv5 {
/// sigp/discv5 노드
discv5: Arc<discv5::Discv5>,
/// RLPx 네트워크의 IpMode
rlpx_ip_mode: IpMode,
/// 체인을 식별하는 키 (예: 'opstack' 또는 'eth')
fork_key: Option<&'static [u8]>,
/// 발견된 피어에 적용되는 필터
discovered_peer_filter: MustNotIncludeKeys,
}

impl Discv5 {
/// 노드를 테이블에 추가
pub fn add_node(&self, node_record: Enr<SecretKey>) -> Result<(), Error> {
let EnrCombinedKeyWrapper(enr) = node_record.into();
```

```
self.discv5.add_enr(enr).map_err(Error::AddNodeFailed)
}

/// 피어와 ID를 금지 목록에 추가
pub fn ban(&self, peer_id: PeerId, ip: IpAddr) {
match discv4_id_to_discv5_id(peer_id) {
Ok(node_id) => {
self.discv5.ban_node(&node_id, None);
self.ban_ip(ip);
}
Err(err) => error!(target: "net::discv5", %err, "failed to ban peer"),
}
}
}
```

DNS Discovery

EIP-1459에 따른 DNS를 통한 노드 발견을 구현합니다.

```
#[derive(Clone, Debug)]
pub struct DnsDiscoveryHandle {
/// 서비스에 명령을 보내는 채널
to_service: UnboundedSender<DnsDiscoveryCommand>,
}
```

```rust
impl DnsDiscoveryHandle {

/// 주어진 링크를 트리에 동기화 시작

pub fn sync_tree(&self, link: &str) -> Result<(), ParseDnsEntryError> {

self.sync_tree_with_link(link.parse()?);

Ok(())

}



/// 발견된 NodeRecord를 스트리밍하는 새 리스너 채널의 수신부 반환

pub async fn node_record_stream(

&self,

) -> Result<ReceiverStream<DnsNodeRecordUpdate>, oneshot::error::RecvError> {

let (tx, rx) = oneshot::channel();

let cmd = DnsDiscoveryCommand::NodeRecordUpdates(tx);

let _ = self.to_service.send(cmd);

rx.await

}

}
```

피어 관리 시스템

PeersManager

네트워크에 알려진 모든 피어의 상태를 유지합니다.

```rust
#[derive(Debug)]
pub struct PeersManager {
/// 네트워크에 알려진 모든 피어
peers: HashMap<PeerId, Peer>,
/// 신뢰할 수 있는 피어 ID 집합
trusted_peer_ids: HashSet<PeerId>,
/// 신뢰할 수 있는 피어의 DNS 이름을 주기적으로 해결하는 리졸버
trusted_peers_resolver: TrustedPeersResolver,
/// 명령 채널의 송신부 복사본
manager_tx: mpsc::UnboundedSender<PeerCommand>,
/// 평판 변경을 가중치로 계산하는 방법
reputation_weights: ReputationChangeWeights,
/// 현재 슬롯 통계 추적
connection_info: ConnectionInfo,
/// 원하지 않는 IP/피어 ID 추적
ban_list: BanList,
/// 현재 백오프된 피어 추적
backed_off_peers: HashMap<PeerId, std::time::Instant>,
}
```

피어 연결 상태 관리

```rust
pub enum PeerConnectionState {
```

```
/// 피어에 대한 활성 연결 없음

Idle,

/// 피어에 대한 연결 시도 중

PendingOut,

/// 피어로부터의 들어오는 연결 대기 중

PendingIn,

/// 피어와 연결됨

Connected,

/// 피어가 연결 해제됨

Disconnected,

}


pub  enum PeerKind {

/// 기본 피어

Basic,

/// 정적으로 구성된 피어

Static,

/// 신뢰할 수 있는 피어

Trusted,

}
```

메시지 전송/수신 프로세스

메시지 타입

```rust
pub  enum PeerMessage<N: NetworkPrimitives> {
/// 새로운 블록 해시 알림
NewBlockHashes(NewBlockHashes),
/// 새로운 블록 알림
NewBlock(NewBlock<N::Block>),
/// 풀링된 트랜잭션
PooledTransactions(NewPooledTransactionHashes),
/// ETH 프로토콜 요청
EthRequest(PeerRequest<N>),
/// 수신된 트랜잭션
ReceivedTransaction(SharedTransactions<N::BroadcastedTransaction>),
/// 트랜잭션 전송
SendTransactions(SharedTransactions<N::BroadcastedTransaction>),
/// 블록 범위 업데이트
BlockRangeUpdated(BlockRangeUpdate),
/// 기타 메시지
Other(ProtocolMessage),
}
```

메시지 처리 플로우

1. **메시지 수신**: 피어로부터 메시지가 도착하면 세션 매니저가 이를 수신합니다.

2. **메시지 분류**: 메시지 타입에 따라 적절한 핸들러로 라우팅됩니다:

- `NewBlockHashes` : 블록 가져오기 프로세스 시작
- `NewBlock` : 블록 검증 및 가져오기
- `PooledTransactions` : 트랜잭션 매니저에 전달
- `EthRequest` : ETH 요청 핸들러에 위임

3. **응답 처리**: 요청에 대한 응답을 생성하고 피어에게 전송합니다.

```
impl<N: NetworkPrimitives> NetworkManager<N> {
/// 피어의 세션으로부터 수신된 메시지 처리
fn on_peer_message(&mut self, peer_id: PeerId, msg: PeerMessage<N>) {
match msg {
PeerMessage::NewBlockHashes(hashes) => {
self.within_pow_or_disconnect(peer_id, |this| {
// 피어의 상태 업데이트
this.swarm.state_mut().on_new_block_hashes(peer_id, hashes.0.clone());
// 해시에 대한 블록 가져오기 프로세스 시작
this.block_import.on_new_block(peer_id, NewBlockEvent::Hashes(hashes));
})
}
PeerMessage::NewBlock(block) => {
self.within_pow_or_disconnect(peer_id, move |this| {
this.swarm.state_mut().on_new_block(peer_id, block.hash);
// 블록 가져오기 프로세스 시작
```

```
this.block_import.on_new_block(peer_id, NewBlockEvent::Block(block));
});
}
PeerMessage::PooledTransactions(msg) => {
self.notify_tx_manager(NetworkTransactionEvent::IncomingPooledTransactionHashes
peer_id,
msg,
});
}
PeerMessage::EthRequest(req) => {
self.on_eth_request(peer_id, req);
}
// ... 기타 메시지 타입 처리
}
}
}
```

네트워크 프로토콜 스택

RLPx 프로토콜

Ethereum의 암호화된 P2P 통신 프로토콜입니다.

```
pub  trait NetworkProtocols: Send + Sync {
```

```rust
/// RLPx 하위 프로토콜 목록에 추가 프로토콜 핸들러 추가
fn add_rlpx_sub_protocol(&self, protocol: RlpxSubProtocol);
}

pub struct RlpxSubProtocol {
/// 프로토콜 이름
pub protocol: ProtocolHandler,
/// 지원되는 버전
pub versions: Vec<u8>,
}
```

세션 관리

```rust
pub enum Direction {
/// 들어오는 연결
Incoming,
/// 특정 노드로의 나가는 연결
Outgoing(PeerId),
}

pub struct PeerInfo {
/// 피어의 발표된 기능
pub capabilities: Arc<Capabilities>,
```

```rust
/// 원격 피어의 식별자
pub remote_id: PeerId,
/// 클라이언트의 이름과 버전
pub client_version: Arc<str>,
/// 피어의 enode
pub enode: String,
/// 연결된 피어의 주소
pub remote_addr: SocketAddr,
/// 세션의 방향
pub direction: Direction,
/// 협상된 eth 버전
pub eth_version: EthVersion,
/// 피어의 연결 종류
pub kind: PeerKind,
}
```

네트워크 이벤트 시스템

이벤트 타입

```rust
pub enum NetworkEvent<R> {
/// 피어 관련 이벤트
Peer(PeerEvent),
```

```
/// 활성 피어 세션

ActivePeerSession {

/// 피어 정보

info: PeerInfo,

/// 요청 전송자

request_tx: PeerRequestSender<R>,

},

}


pub  enum PeerEvent {

/// 새로운 피어가 연결됨

SessionEstablished(PeerInfo),

/// 피어 연결이 끊어짐

SessionClosed {

peer_id: PeerId,

reason: Option<DisconnectReason>,

},

}
```

이벤트 처리

```
impl<N: NetworkPrimitives> NetworkEventListenerProvider for NetworkHandle<N> {

type Primitives = N;
```

```
fn event_listener(&self) -> EventStream<NetworkEvent<PeerRequest<Self::Primitiv
self.inner.event_sender.new_listener()
}

fn discovery_listener(&self) -> UnboundedReceiverStream<DiscoveryEvent> {
let (tx, rx) = mpsc::unbounded_channel();
let _ = self.manager().send(NetworkHandleMessage::DiscoveryListener(tx));
UnboundedReceiverStream::new(rx)
}
}
```

네트워크 보안 및 평판 시스템

평판 관리

```
pub enum ReputationChangeKind {
/// 나쁜 메시지
BadMessage,
/// 나쁜 블록
BadBlock,
/// 나쁜 트랜잭션
BadTransactions,
```

```
/// 유용한 헤더
UsefulHeader,
/// 유용한 블록
UsefulBlock,
/// 연결 실패
FailedToConnect,
/// 시간 초과
Timeout,
/// 연결 해제
Disconnect,
/// 기타
Other(i32),
}

impl NetworkHandle<N> {
/// 주어진 피어에 대한 평판 변경 전송
pub fn reputation_change(&self, peer_id: PeerId, kind: ReputationChangeKind) {
self.send_message(NetworkHandleMessage::ReputationChange(peer_id, kind));
}
}
```

금지 목록 관리

```rust
impl Discv4 {
/// 피어와 ID를 금지 목록에 추가
pub fn ban(&self, node_id: PeerId, ip: IpAddr) {
let cmd = Discv4Command::Ban(node_id, ip);
self.send_to_service(cmd);
}

/// IP를 금지 목록에 추가
pub fn ban_ip(&self, ip: IpAddr) {
let cmd = Discv4Command::BanIp(ip);
self.send_to_service(cmd);
}

/// 피어를 금지 목록에 추가
pub fn ban_node(&self, node_id: PeerId) {
let cmd = Discv4Command::BanPeer(node_id);
self.send_to_service(cmd);
}
}
```

설정 및 커스터마이징

네트워크 설정

```rust
pub struct NetworkConfig<C, N: NetworkPrimitives> {
/// 블록체인 클라이언트
pub client: C,
/// 세션 인증에 사용되는 비밀 키
pub secret_key: SecretKey,
/// Discovery v4 주소
pub discovery_v4_addr: Option<SocketAddr>,
/// Discovery v4 설정
pub discovery_v4_config: Option<Discv4Config>,
/// Discovery v5 설정
pub discovery_v5_config: Option<reth_discv5::Config>,
/// 들어오는 연결을 수신할 주소
pub listener_addr: SocketAddr,
/// 피어 설정
pub peers_config: PeersConfig,
/// 세션 설정
pub sessions_config: SessionsConfig,
/// 체인 ID
pub chain_id: u64,
/// 네트워크 모드 (PoW 또는 PoS)
pub network_mode: NetworkMode,
/// 부트 노드
```

```
pub boot_nodes: Vec<NodeRecord>,
}
```

피어 설정

```
pub struct PeersConfig {
/// 슬롯을 다시 채우는 간격
pub refill_slots_interval: Duration,
/// 연결 정보
pub connection_info: ConnectionsConfig,
/// 평판 가중치
pub reputation_weights: ReputationChangeWeights,
/// 금지 기간
pub ban_duration: Duration,
/// 백오프 기간
pub backoff_durations: PeerBackoffDurations,
/// 신뢰할 수 있는 노드만 연결할지 여부
pub trusted_nodes_only: bool,
}
```

관련 컴포넌트

의존성 관계

- **Discovery**: 피어 발견을 위해 Discv4, Discv5, DNS Discovery 사용
- **Session Management**: RLPx 프로토콜을 통한 암호화된 세션 관리
- **Transaction Pool**: 트랜잭션 전파 및 동기화
- **Block Import**: 새로운 블록 수신 및 검증
- **Storage**: 피어 정보 및 네트워크 상태 저장

상호작용 방식

1. **Discovery → Peers**: 발견된 피어를 피어 매니저에 추가
2. **Peers → Sessions**: 연결할 피어를 세션 매니저에 전달
3. **Sessions → Protocol**: 설정된 세션을 통해 프로토콜 메시지 교환
4. **Protocol → Application**: 수신된 메시지를 적절한 애플리케이션 컴포넌트로 라우팅

이러한 구조를 통해 Reth는 안정적이고 확장 가능한 P2P 네트워킹 시스템을 제공하며, Ethereum 네트워크의 분산 특성을 효과적으로 지원합니다.